Classification Mini-Project: Telco Customer Churn Prediction & Deployment

Project Overview

Imagine you are working as a Machine Learning Engineer for a telecommunications company. Your goal is to build an end-to-end intelligent system that predicts whether a customer will cancel their subscription (`Churn = Yes`) or stay with the company (`Churn = No`). This helps the company design retention programs before they lose customers.

Dataset

- **Telco Customer Churn Dataset** (uploaded on drive)
- It includes customer demographic info, services they signed up for (internet, phone, tech support, etc.), and account information (contract type, monthly charges, tenure).

Required Project Steps

Step 1: Data Preprocessing

- Handle missing values or hidden spaces.
- Convert categorical (text) columns into numeric values using proper encoding.
- Split the data into Training and Testing sets, and apply feature scaling.

Step 2: Baseline Training & Model Comparison

Train **all 7 models** listed below on your dataset and compare their performance to find the best one:

1. Logistic Regression
2. Support Vector Machine (SVM)
3. K-Nearest Neighbors (KNN)
4. Decision Tree (DT)
5. Random Forest (RF)
6. AdaBoost
7. XGBoost

- **Evaluation & Metric Selection:**
    - Do not just rely on `Accuracy`. Think critically about the nature of the Churn dataset (which is often imbalanced, as most customers stay and fewer leave).

- **Choose the right evaluation metric(s)** (such as Precision, Recall, F1-Score, or Confusion Matrix) that best suit this business problem.
- **Explain why:** In your report/code comments, briefly justify **why** you chose this specific metric and why Accuracy alone might be misleading here.

Step 3: Model Fine-Tuning

- Take the model that achieved the best performance and apply **Hyperparameter Tuning** (using `GridSearchCV` or manual tuning) to optimize its settings and maximize accuracy.

Step 4: Save the Model

- Save your final, optimized model to your computer using **`joblib`** or **`pickle`** (e.g., `best_model.pkl`).

Step 5: Build a Streamlit Web App (UI)

- Create an interactive web app using **`Streamlit`**.
- Load your saved model inside the app.
- Add input fields for customer details (e.g., contract type, monthly charges, tenure).
- Add a **Predict** button so that when a user clicks it, the app runs the model and instantly shows the result: *(e.g., "This customer is at risk of leaving ⚠️" or "This customer is likely to stay ✅")*.

Project Deliverables (What you need to submit):

1. **Clean Code / Notebook:** The script or Jupyter Notebook showing data preprocessing, training, and tuning.
2. **Saved Model File:** The `.pkl` or `.joblib` model file.
3. **Streamlit App File:** The `app.py` file to run the web interface.
4. **Short Video Demo:** A brief video recording (1 minutes) showing how to run your Streamlit app, inputting sample customer data, and seeing the live prediction in action.